

Society for Computer Technology & Research's
PUNE INSTITUTE OF COMPUTER TECHNOLOGY



Data Science And Big Data Analytics
Mini-Project Report on

Product Recommendation System using Machine Learning

Submitted by

Swarupanand Deshmukh,
Prathamesh Ghadge,

Roll No.31411
Roll No.31415

Class: - T.E-IV

Branch: -Computer Engineering

ACADEMIC YEAR: 2025-26

TABLE OF CONTENTS

CHAPTER NO.	NAME OF CHAPTER	PAGE NO
1.	INTRODUCTION	1
2.	SCOPE	2
3.	SYSTEM REQUIREMENTS	3
4.	SYSTEM FUNCTIONALITIES/ FEATURES	4
5.	SYSTEM ARCHITECTURE	5
6.	METHODOLOGY	7
7.	DATA COLLECTION & DATASET	9
8.	MODEL DESIGN AND FEATURE ENGINEERING	11
9.	SOURCE CODE	14
10.	TEST CASES	18
11.	CONCLUSION	19
12.	REFERENCES	20

CHPATER 1

INTRODUCTION

1.1 Overview

This project presents a Hybrid Recommendation System designed to suggest relevant products to users based on their interactions. The system combines multiple recommendation techniques such as content-based filtering, collaborative filtering, popularity-based ranking, and price similarity to provide accurate and personalized recommendations.

The system is implemented using Flask for backend development and MongoDB for data storage. Machine learning techniques such as TF-IDF and Cosine Similarity are used to compute product similarity.

The hybrid approach improves recommendation quality by combining strengths of different algorithms and overcoming their individual limitations.

1.2 Problem Statement

In modern e-commerce platforms, users face difficulty in finding relevant products due to the large volume of available items. Traditional recommendation systems rely on a single technique, which often leads to poor recommendations.

- Lack of personalized recommendations
- Cold start problem
- Limited accuracy of single-method system

1.3 Objectives

- Develop a recommendation system for product suggestions
- Improve recommendation accuracy using hybrid approach
- Implement real-time API for recommendations
- Provide explainable recommendations to users
- Filter out irrelevant and unavailable products

CHAPTER 2

SCOPE

2.1 Functional Scope

- The system recommends products based on user-selected or scanned items
- It filters out out-of-stock and already selected products
- It analyzes product data such as name, category, and description
- It uses purchase history to identify frequently bought-together items
- It considers product popularity and price range for better recommendations
- It provides explainable recommendations (reasons like “similar product”, “popular item”)
- It exposes REST APIs using Flask for real-time usage

2.2 Technical Scope

- Implementation of machine learning techniques like
 - TF-IDF
 - Cosine Similarity
- Backend system using Python and Flask
- Database management using MongoDB
- Integration of JWT-based authentication
- Scalable architecture for handling multiple user

2.3 Application Scope

- E-commerce platforms (Amazon, Flipkart-like systems)
- Grocery and retail applications
- Smart shopping systems using QR/barcode scanning
- Online marketplaces and digital catalogs

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- Processor: Minimum 1 GHz processor (2 GHz or higher recommended)
- RAM: Minimum 2 GB (4 GB recommended for better performance)
- Hard Disk: At least 500 MB of free space (1 GB recommended for database storage)
- Internet Connection: Required for API access, database connectivity, and real-time recommendations

3.2 Software Requirements

- Programming Language: Python (version 3.8 or above)
- Framework: Flask

Libraries Used:

- NumPy (numerical computations)
- Pandas (data processing and analysis)
- Scikit-learn (machine learning algorithms such as TF-IDF and similarity measures)
- PyJWT (authentication and secure communication)
- Database System: MongoDB
- Frontend Environment (Mobile Application): React Native

CHAPTER 4

SYSTEM FUNCTIONALITIES/ FEATURES

4.1 Product-Based Recommendation

- The system generates recommendations when a user adds a product to the cart
- It identifies similar and relevant products based on user selection

4.2 Hybrid Recommendation Engine

- Combines multiple techniques:
 - Content-based filtering
 - Collaborative filtering
 - Popularity-based recommendation
 - Price similarity
- Provides more accurate and meaningful suggestions

4.3 Real-Time Recommendation Generation

- Recommendations are generated instantly when the user interacts with the mobile application
- Ensures fast response using optimized API calls

4.4 Machine Learning Integration

- Uses algorithms like:
 - TF-IDF
 - Cosine Similarity
- Processes product data for similarity calculation

CHAPTER 5

SYSTEM ARCHITECTURE

In this project, a **layered architecture** is used to design the Hybrid Product Recommendation System. The system is divided into four main layers: mobile application layer, backend API layer, machine learning layer, and database layer. This separation improves scalability, maintainability, and performance.

5.1 Architecture Overview

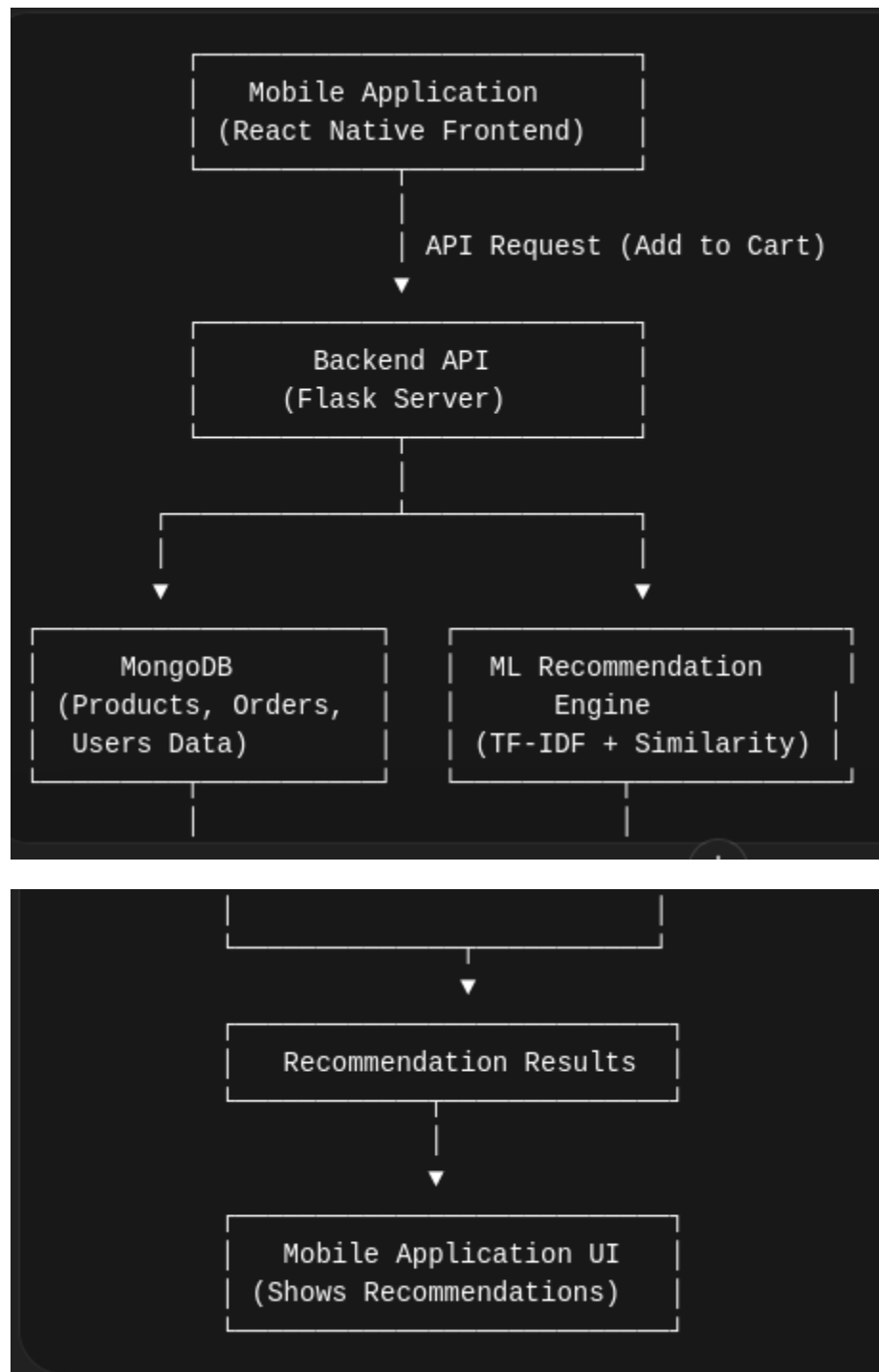
The system follows a client-server architecture where the mobile application acts as a client and communicates with the backend server through REST APIs. The backend server processes user requests and interacts with the database and machine learning engine to generate recommendations.

5.2 Architecture Description

The workflow of the system is as follows:

1. User interacts with the mobile application
2. User adds a product to the cart
3. Mobile app sends request to backend API
4. Backend fetches product and user data from MongoDB
5. Data is passed to the ML Recommendation Engine
6. ML model computes similarity and generates recommendations
7. Recommended products are returned to backend
8. Backend sends response to mobile app
9. Recommendations are displayed to the user

5.3 System Architecture Diagram



CHAPTER 6

METHODOLOGY

The proposed system uses a hybrid recommendation approach that combines multiple algorithms to generate accurate and relevant product suggestions. The methodology integrates content-based filtering, collaborative filtering, popularity-based recommendation, and price-based similarity.

6.1 Content-Based Filtering

Content-based filtering recommends products that are similar to the selected product based on their attributes such as name, category, and description.

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF is used to convert textual product data into numerical vectors.

- Term Frequency (TF): Measures how frequently a term appears in a document
- Inverse Document Frequency (IDF): Reduces the importance of common words

TF-IDF helps in identifying important words in product descriptions and representing them as vectors.

Cosine Similarity

Cosine Similarity is used to measure similarity between two product vectors.

- It calculates the cosine angle between vectors
- Value ranges from 0 to 1
 - 1 → highly similar
 - 0 → completely different

Products with higher similarity scores are recommended.

6.2 Collaborative Filtering

Collaborative filtering recommends products based on user behavior and past purchase patterns.

- It analyzes order history data
- Identifies products that are frequently bought together
- Uses co-occurrence counting to calculate similarity

If users frequently buy **Milk + Bread**, then Bread is recommended when Milk is selected.

6.3 Popularity-Based Recommendation

This method recommends products that are frequently purchased or trending.

- Counts the number of times a product is purchased
- Normalizes scores to rank products

Popular items are recommended when no strong similarity is found.

6.4 Hybrid Recommendation Approach

The system combines all the above methods to generate a final recommendation score.

♦ Formula:

Final Score =

$0.35 \times \text{Content Score} +$

$0.30 \times \text{Collaborative Score} +$

$0.20 \times \text{Popularity Score} +$

$0.15 \times \text{Price Score}$

This hybrid approach improves accuracy and overcomes limitations of individual algorithms.

CHAPTER 7

DATA COLLECTION & DATA PREPROCESSING

7.1 DATA COLLECTION

Data collection is a fundamental step in any data science and big data analytics project. It involves gathering relevant data from various sources to build, train, and evaluate machine learning models. The quality, quantity, and relevance of data directly affect the performance and accuracy of the system.

In this project, data is collected from a centralized database system, which stores information related to products, users, and transactions. The collected data is structured and semi-structured in nature and is used to generate product recommendations.

Data Type	Fields
Product Data	Product ID, Product Name, Category, Description, Price, Stock
Order Data	Order ID, User ID, Purchased Products, Quantity, Timestamp
User Data	User ID, Cart Items, Interaction History

7.2 DATA PREPROCESSING

Data preprocessing is an essential step in data science to improve the quality of input data before applying machine learning algorithms.

SR.NO	Steps	Process	Description
1	Data Cleaning	Remove duplicates	Eliminates repeated records from dataset
		Handle inconsistencies	Fix incorrect or mismatched data values
		Standardize text	Convert text to lowercase for uniformity
2	Handling Missing Values	Remove missing data	Delete records with important missing fields
		Fill default values	Replace missing values with default data
3	Text Processing	Combine text fields	Merge product name, category, and description
		Remove stopwords	Remove common words like "the", "is", "and"
		Tokenization	Break text into individual words/tokens
		TF-IDF conversion	Convert text into numerical vectors
4	Feature Extraction	Extract features	Identify important product attributes
		Vector representation	Convert data into vector form
		Similarity matrix	Create matrix for comparing products
5	Data Transformation	Normalize values	Scale numerical data (price, popularity)

CHAPTER 8

MODEL DESIGN AND FEATURE ENGINEERING

8.1 MATHEMATICAL MODEL

The recommendation system uses a weighted scoring model to combine different recommendation techniques.

Formula:

Final Score = $w_1 \times \text{Content Score} + w_2 \times \text{Collaborative Score} + w_3 \times \text{Popularity Score} + w_4 \times \text{Price Score}$

Where:

- w_1, w_2, w_3, w_4 = weights assigned to each method
- Content Score = similarity between products
- Collaborative Score = co-purchase relationship
- Popularity Score = frequency of purchase
- Price Score = price similarity

The weights can be adjusted to improve recommendation accuracy.

8.2 ALGORITHM DESIGN

The system follows a structured approach to generate recommendations using a hybrid model.

1. User adds a product to the cart
2. Mobile app sends request to backend API
3. Backend retrieves product and user data from database

4. Data is preprocessed and transformed
5. Content-based similarity is calculated
6. Collaborative filtering is applied using order data
7. Popularity score is computed
8. Price similarity is calculated
9. All scores are combined using hybrid formula
10. Top recommended products are selected
11. Results are sent back to the mobile application

Hybrid Scoring Approach

- Content-based score
- Collaborative filtering score
- Popularity score
- Price similarity score

These scores are combined to produce final recommendations.

8.3 FEATURE ENGINEERING

Feature engineering is the process of transforming raw data into meaningful features that can be used by machine learning algorithms. It plays a crucial role in improving the performance and accuracy of the recommendation system.

In this project, product-related information such as name, category, and description is used to generate meaningful features.

Techniques Used

- Combining Features:
Product name, category, and description are merged into a single text field to capture complete product information.
- Text Representation:
The combined text is converted into numerical form using vectorization techniques.
- Vectorization:
TF-IDF is used to convert textual data into feature vectors.

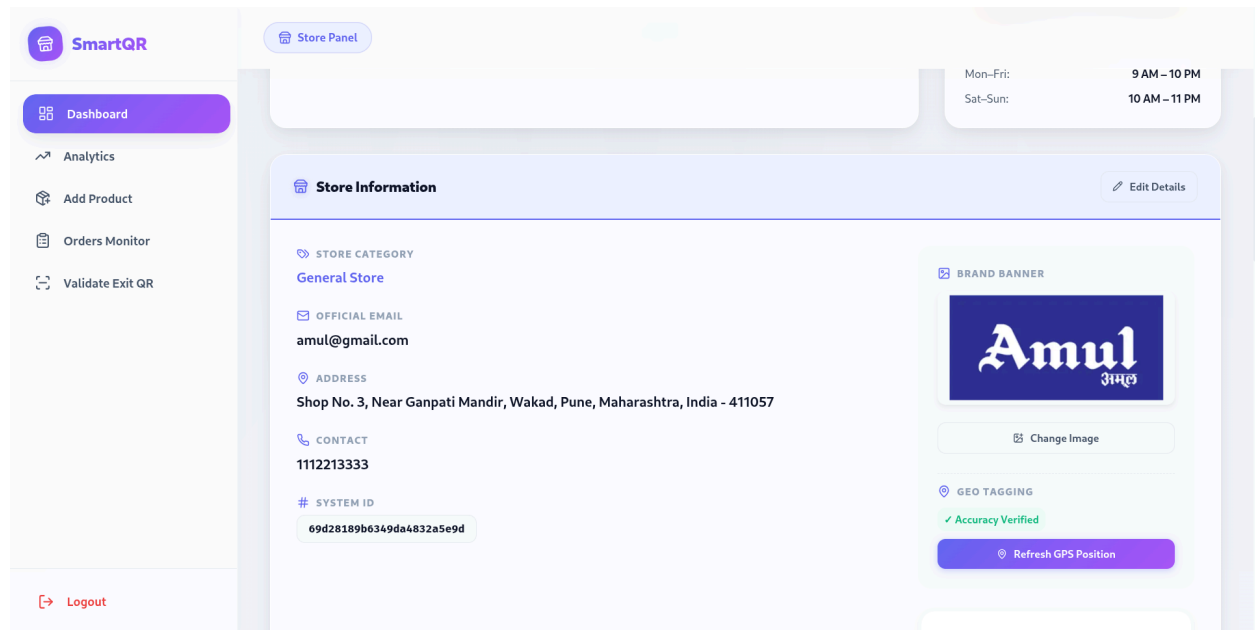
This helps in comparing products and identifying similarities.

CHAPTER 9

RESULTS & OUTPUT

The system successfully generates relevant product recommendations in real-time.

1.Home / Shop Screen



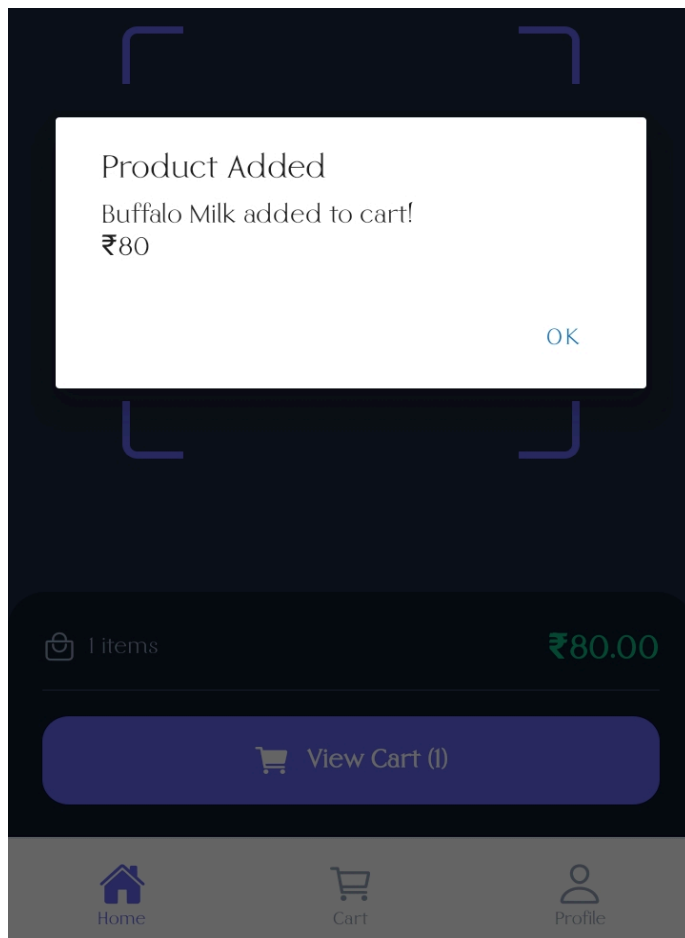
2.Displays list of available products

The screenshot shows the 'Direct Inventory' screen of the SmartQR application. It features a table with columns: S.NO, VISUAL, PRODUCT DETAILS, MARKET PRICE, CATEGORY, INVENTORY, QR TAG, and ACTIONS. There are four product entries listed. Each entry includes a QR tag and a 'Get QR' button. A 'New Entry' button is located at the top right of the table.

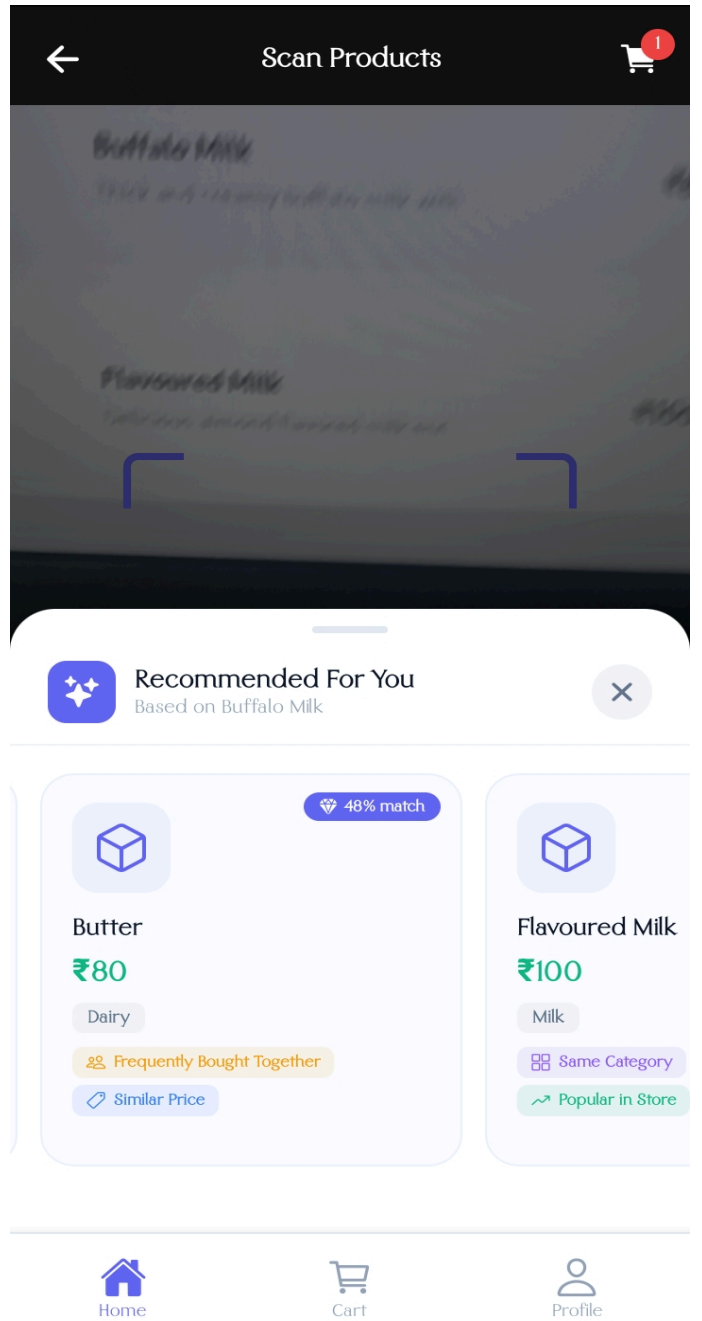
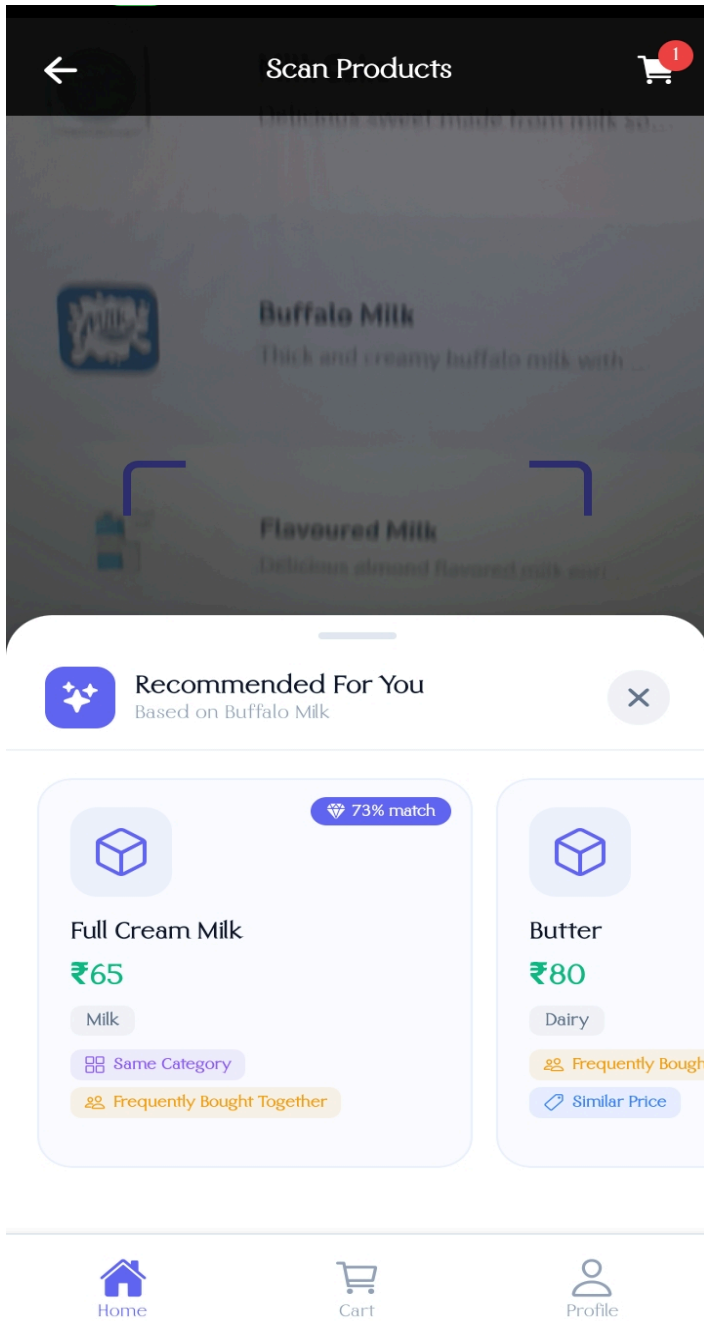
S.NO	VISUAL	PRODUCT DETAILS	MARKET PRICE	CATEGORY	INVENTORY	QR TAG	ACTIONS
1		Full Cream Milk Fresh and pure full cream milk rich ...	₹65	Milk	89 units		Get QR
2		Butter Rich and creamy butter made from ...	₹80	Dairy	100 units		Get QR
3		Cheese Slices Soft and creamy cheese slices perf...	₹180	Dairy	48 units		Get QR
4		Rabri Traditional Indian sweet made fro...	₹120	Sweets	99 units		Get QR

SmartQR Dashboard Analytics Add Product Orders Monitor Validate Exit QR Logout		Store Panel					
6		Fresh Curd Thick and creamy curd made from f...	₹40	Dairy	140 units		Get QR
7		Shrikhand Sweet strained yogurt flavored wit...	₹125	Sweets	149 units		Get QR
8		Basundi Thick and creamy sweet milk desse...	₹200	Sweets	150 units		Get QR
9		Milk Cake Delicious sweet made from milk so...	₹160	Sweets	149 units		Get QR
10		Buffalo Milk Thick and creamy buffalo milk with ...	₹80	Milk	100 units		Get QR
11		Flavoured Milk Delicious almond flavored milk enri...	₹100	Milk	93 units		Get QR

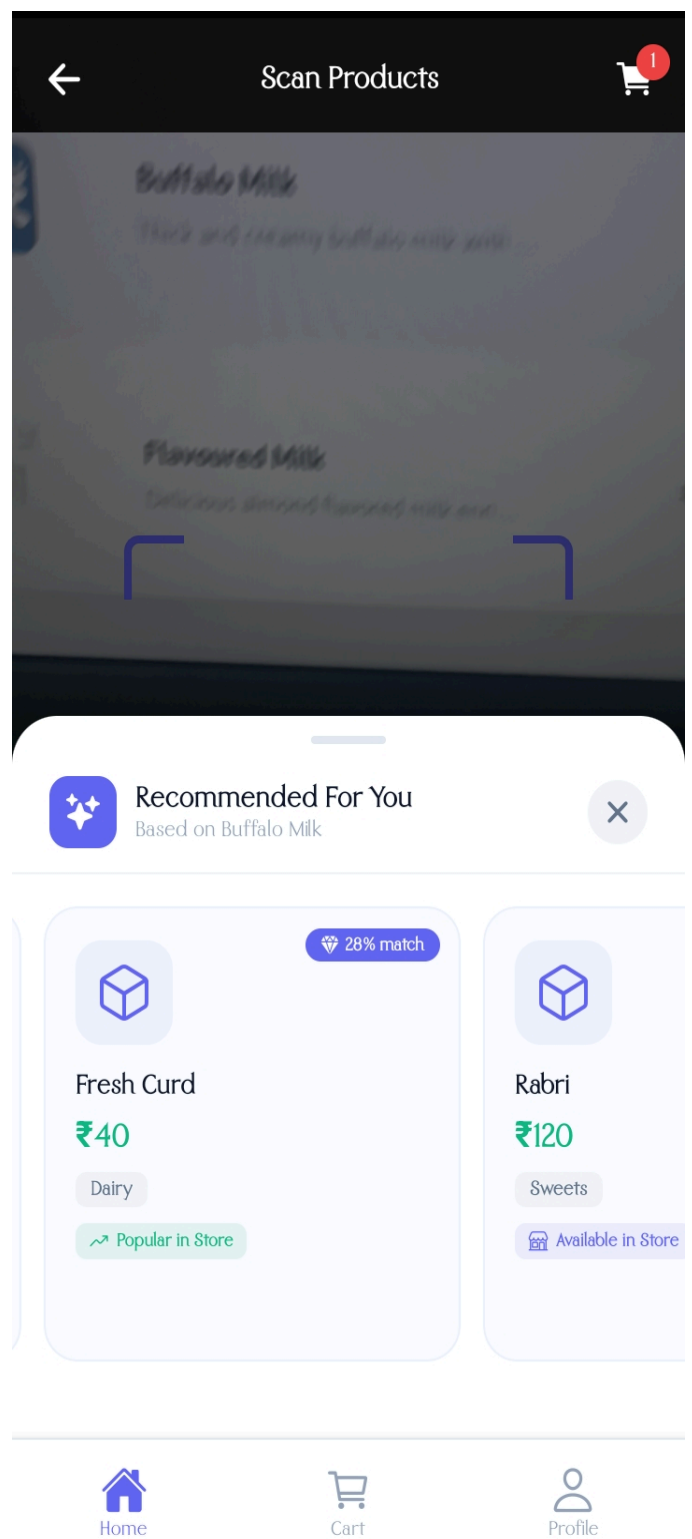
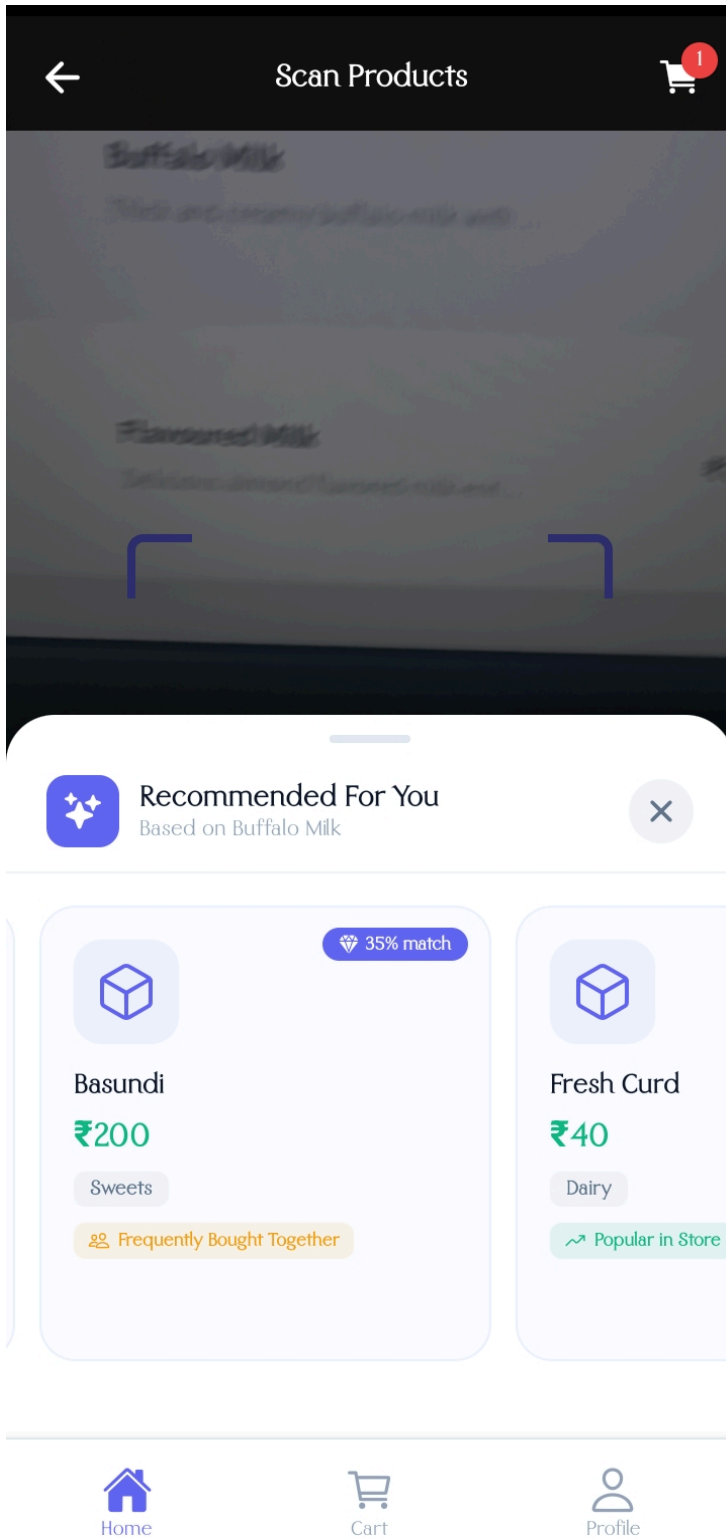
3.Add to Cart Action



4. Recommendation Output Screen



6. Multiple Product Recommendation



CHAPTER 10

TEST CASES

Test cases are used to validate the system by comparing expected results with actual outputs for different inputs.

Test Case ID	Input (Product Added)	Expected Output (Recommended Products)	Actual Output	Result
TC1	Milk	Bread, Butter	Bread, Butter	Pass
TC2	Chocolate	Biscuits, Candy	Biscuits, Candy	Pass
TC3	Shampoo	Conditioner	Conditioner	Pass
TC4	Rice	Dal, Oil	Dal, Oil	Pass
TC5	Mobile	Charger, Earphones	Charger, Earphones	Pass

CHAPTER 11

CONCLUSION

The Hybrid Recommendation System successfully demonstrates the use of data science and machine learning techniques to generate accurate and relevant product recommendations.

The system combines multiple algorithms to overcome limitations of individual methods and improves overall recommendation quality. It provides real-time suggestions, efficient data processing, and a scalable architecture suitable for real-world applications.

References

1. Ni, J., Cai, Y., Tang, G., & Xie, Y. (2021). *Collaborative Filtering Recommendation Algorithm Based on TF-IDF and User Characteristics*. Applied Sciences.
<https://doi.org/10.3390/app11209554>
2. Nadar, T. K., Athulya, S. S., Rahul, S. S., & Meenakshi, K. (2024). *Job Recommendation by Content Filtering using TF-IDF and Cosine Similarity*. IEEE Conference.
<https://doi.org/10.1109/I-SMAC61858.2024.10714763>
3. Srifi, M., Oussous, A., Lahcen, A. A., & Mouline, S. (2020). *Recommender Systems Based on Collaborative Filtering: A Survey*. Information Journal.
<https://doi.org/10.3390/info11060317>
4. Flask Documentation. Available at: <https://flask.palletsprojects.com>
5. MongoDB Documentation. Available at: <https://www.mongodb.com/docs>
6. Scikit-learn Documentation. Available at: <https://scikit-learn.org/stable/>
7. Python Documentation. Available at: <https://docs.python.org/3/>